# Introduction to Ordered Set Pattern

## What is an Ordered Set?

Ordered Set is an extension of the Set data structure.

An **Ordered Set** is a powerful data structure that blends the uniqueness of a set with the ordered sequence of a list. Unlike traditional sets, where the elements are unsorted, an ordered set maintains the sequence of its elements, making it a versatile choice for many applications.

The primary purpose of using an ordered set is to have a collection that not only avoids duplicate elements but also preserves their order. This ordering can be either the natural ordering of the elements or defined by a custom comparator. The ordered set is particularly useful when the sequence in which data is processed or retrieved is crucial.

## Ordered Set vs. Set: The Difference

The primary difference between a standard set and an ordered set is the element ordering. In a regular set, elements are stored with no particular order, and iterating over a set does not guarantee the order in which elements are returned. In contrast, an ordered set maintains its elements in a sorted order, whether it's natural ordering or a specified one.

The ordered set can perform all operations performed by the set data structure. Additionally, it can perform below two operations in $O(logN)$ time.

1. **find_by_order(k)**: It efficiently retrieves the kth element in the set, measured from zero, in $O(logN)$ time. It's a game-changer for direct access based on element order. For example, in a set `{1, 5, 6, 17, 88}`:
    - `find_by_order(2)` fetches the 3rd element in the set, which is 6.
    - `find_by_order(4)` accesses the 5th element in the set, i.e., 88.
2. **order_of_key(k)**: This operation determines the number of elements in the set that are strictly smaller than the specified element k, also in O(log n) time. It's a quick way to gauge the position of an element in the sorted order. Considering the same set `{1, 5, 6, 17, 88}`:
    - `order_of_key(6)` reveals that 2 elements are smaller than 6.
    - `order_of_key(25)` shows that 4 elements are smaller than 25.

These advanced functions extend the capabilities of ordered sets, making them a more robust and versatile choice compared to traditional sets, especially in scenarios requiring both order-sensitive and unique data management.

## Practical Applications of Ordered Set with Examples

Ordered sets can be used in scenarios where data is constantly updated and specific range-based queries are required. Let's explore some examples to demonstrate the utility of ordered sets in managing dynamic data sets and executing range queries effectively.

### Example 1: Managing and Querying a Dynamically Updated Array

Imagine a system where data elements are continuously added to an array. After each new addition, a query is made to find out how many elements within the array fall between a specified range ([l, r]). Initially, the array is empty.

#### Role of an Ordered Set

In such a situation, an ordered set is invaluable. It can keep the elements in a sorted sequence as they are added, facilitating quick and accurate range queries.

To illustrate, let's look at the below example:

```
Input: 5
       [1, 2]
       1
       [2, 5]
       2
       [1, 5]

Output: 0
        1
        3
```

**Explanation**:

- The array starts off as empty.
- Post inserting `5`, the query for range `[1, 2]` yields no matching elements.
- On adding `1`, within the range `[2, 5]`, the element `5` meets the criteria.
- With `2` added, all elements (`1, 2, 5`) fall within the queried range `[1, 5]`.

Through this example, the ordered set's ability to manage and query data in a dynamic environment is evident. Its ordered nature allows for efficient calculation of element counts within specific ranges, showcasing its practicality in scenarios with frequent data updates and the need for immediate, accurate range-based information.

### Example 2: User Activity Logs

Imagine an application that maintains a log of user activities. An ordered set can be used to keep these logs in a chronological sequence without any duplicates, ensuring efficient data retrieval in the exact order of occurrence.

## Implementing Ordered Set

Here's a basic implementation of an ordered set:

Java

```java
import java.util.*;

class OrderedSetJava {
    private TreeSet<Integer> set;

    public OrderedSetJava(Comparator<Integer> comparator) {
        set = new TreeSet<>(comparator);
    }

    public void add(int element) {
        set.add(element);
    }

    public Integer find_by_order(int k) {
        if (k < 0 || k >= set.size()) return null;
        return (Integer)set.toArray()[k];
    }

    public int order_of_key(int element) {
        return set.headSet(element, true).size();
    }
```

Run Code Result Stdout

You must run your code first

Execute

## Time and Space Complexity Analysis

- **Time complexity**: The time complexity for common operations like addition, removal, and searching in a `TreeSet` is O(log n), where n is the number of elements in the set. This is because `TreeSet` is typically implemented using a tree structure.
- **Space complexity**: The space complexity is O(n), as it needs to store each unique element.

Let's jump onto our first problem and apply the **Ordered set** pattern.

---

← Previous
Solution: Path With Minimum Effort

Next →
Merge Similar Items (Easy)

☐ Mark as Completed